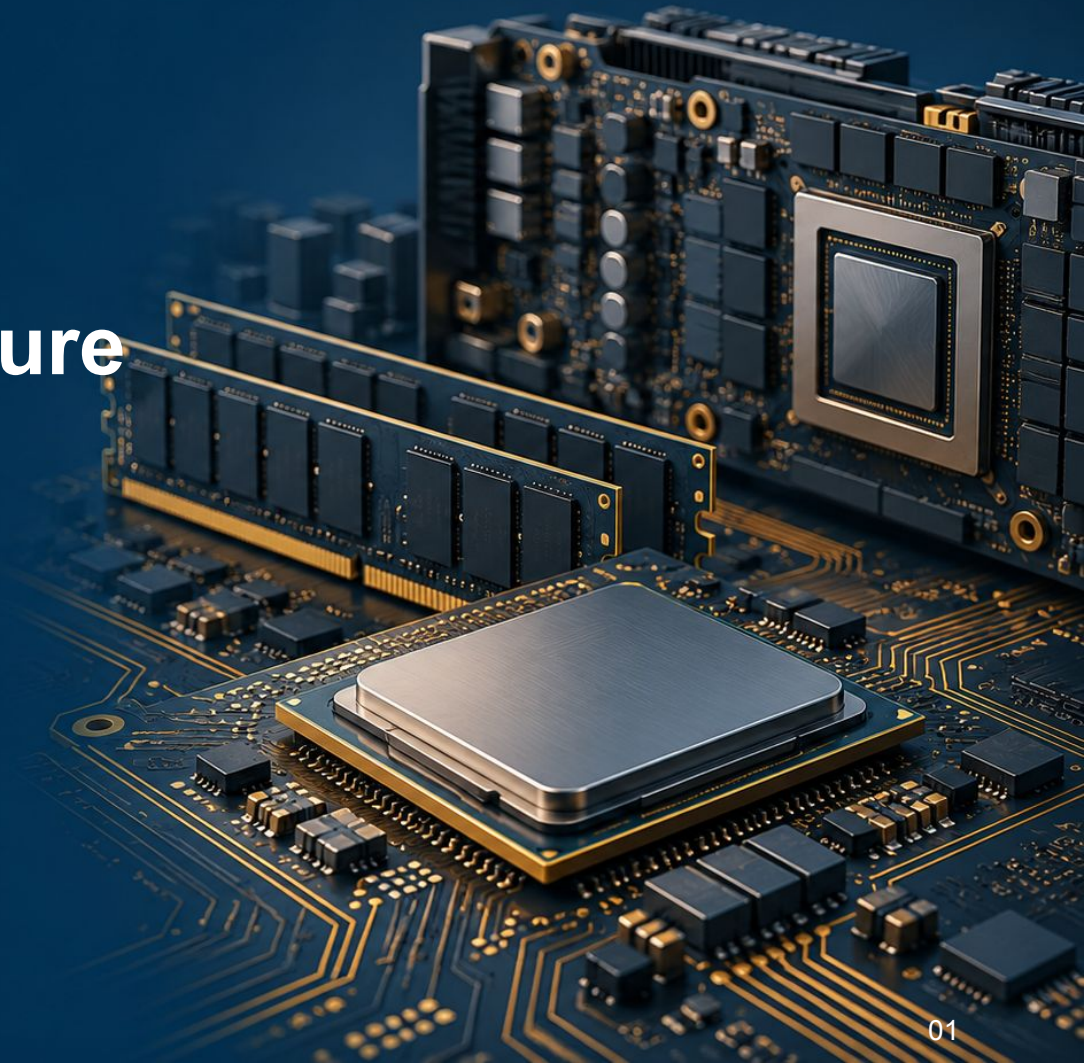
UC BERKELEY MFE

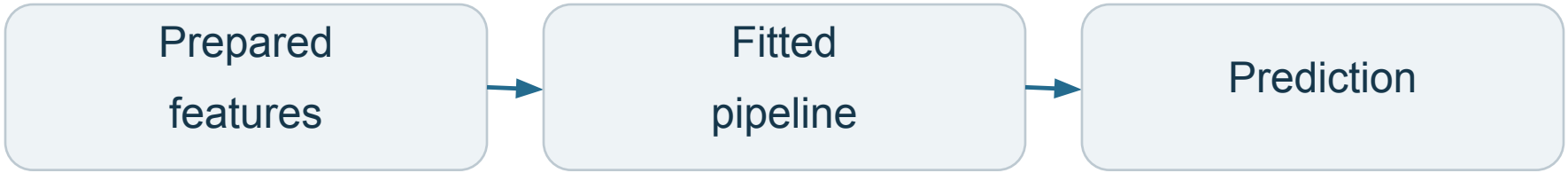
Systems architecture and production engineering

Lesson 9

Aneesh Sachdeva
November 2, 2026



The system around a model



Last week: features, training, evaluation, backtesting.

Today: clear inputs, repeatable execution, other callers, visible failures.

Today’s lesson

Debugging and code quality

Testing

APIs and systems architecture

Containers

Continuous integration, deployment, and operation

A deployment failure at Knight Capital

1 2 3 4 5 6 7 **8**

Seven servers received the new code; one did not.

A reused flag activated old behavior on the missed server.

Losses exceeded $460 million on August 1, 2012.

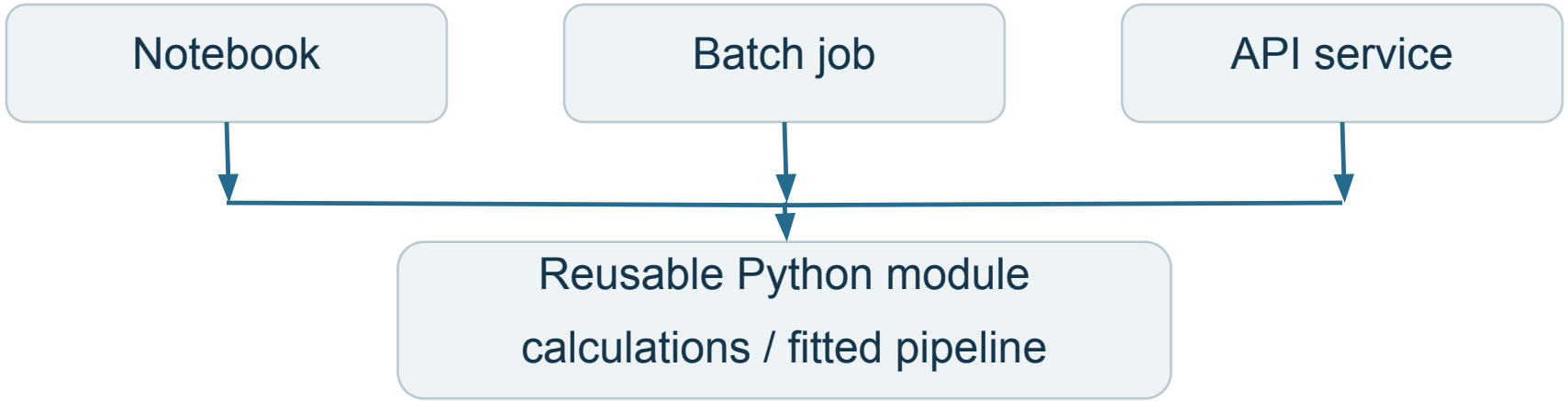
Verify the deployed version and watch the system after release.

DEBUGGING

Debugging and code quality

A repeatable failure gives you evidence to work with.

Notebooks, modules, and services



Keep the reusable calculation independently testable.

A notebook remains useful for research and presentation.

A systematic debugging process

1. Observe and reproduce

Save the exact input and result.

Make the failure repeatable.

2. Isolate the cause

Reduce the input and code.

Inspect intermediate values.

3. Test an explanation

Change one assumption.

Compare expected and actual.

4. Fix and verify

Keep a regression test.

Check nearby behavior.

A calculation that runs but is wrong

2 shares × $100 × 10 basis points

10 basis points = 0.1% = 0.001

Expected fee = $0.20

```
return shares * price * fee_bps / 100
```

This line returns $20. Where is the unit conversion wrong?

Formatting, linting, and type checking

Formatting

Makes code layout consistent.

Ruff format

Linting

Flags likely mistakes such as an unused import.

Ruff check

Static type checking

Flags incompatible uses before execution.

mypy is one option.

Behavior tests

Check a specified result.

Our fee bug needs this check.

Local checks before a pull request

```
uv run ruff format --check .
uv run ruff check .
uv run pytest -q
```

Read the failing check and its file/line.

Fix the cause, rerun, then propose the change.

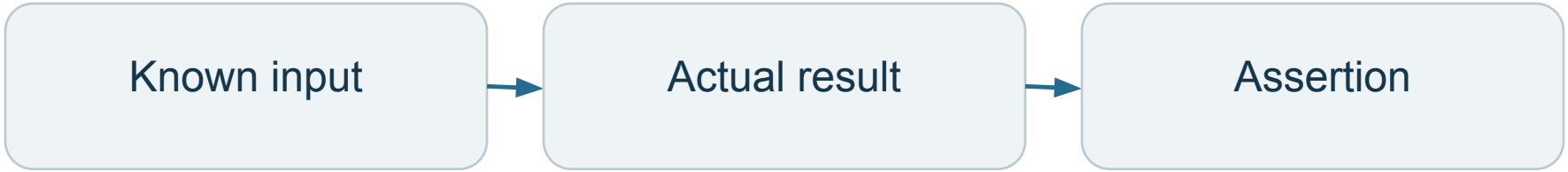
A Makefile can give these commands short names.

TESTING

Testing

Executable checks compare behavior with an independent expectation.

What a test checks

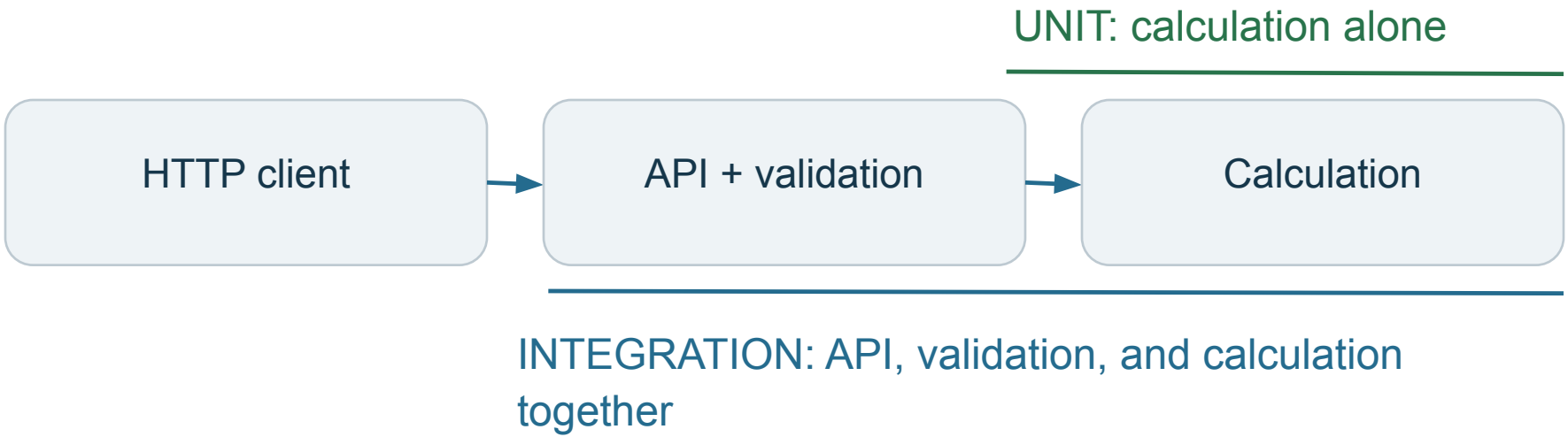


Expected result: $0.20, calculated independently.

Actual result: produced by the function.

Assertion: do they agree within the chosen tolerance?

Unit, integration, and end-to-end tests



END TO END: client → running container → response

A pytest test, line by line

```
def test_known_fee(fee_function):
    actual = fee_function(
        shares=2, price=100, fee_bps=10
    )
    assert actual == pytest.approx(0.20)
```

Arrange: inputs Act: call Assert: compare

Buggy version: $20 → corrected version: $0.20

Boundary cases and invalid inputs

Contract: nonnegative quantity and fee; positive price.

Finite inputs only. Fractional shares are allowed.

Zero quantity	0 fee
Zero fee rate	0 fee
0.5 shares, $100, 10 bps	$0.05
Negative price / NaN	Reject with ValueError

Repeatable tests and controlled dependencies

```
@pytest.fixture
def valid_input():
    return {"momentum_20d": 0.02,
            "volatility_20d": 0.15}
```

Fixture: supplies a known test input.

Mock: substitutes a dependency’s behavior.

Example: force model.predict to fail; verify a 500 response.

Software tests and model evaluations

Software tests

Does invalid input get rejected?

Are features in the correct order?

Does the API match direct Python?

Does saved preprocessing stay fixed?

Model evaluations

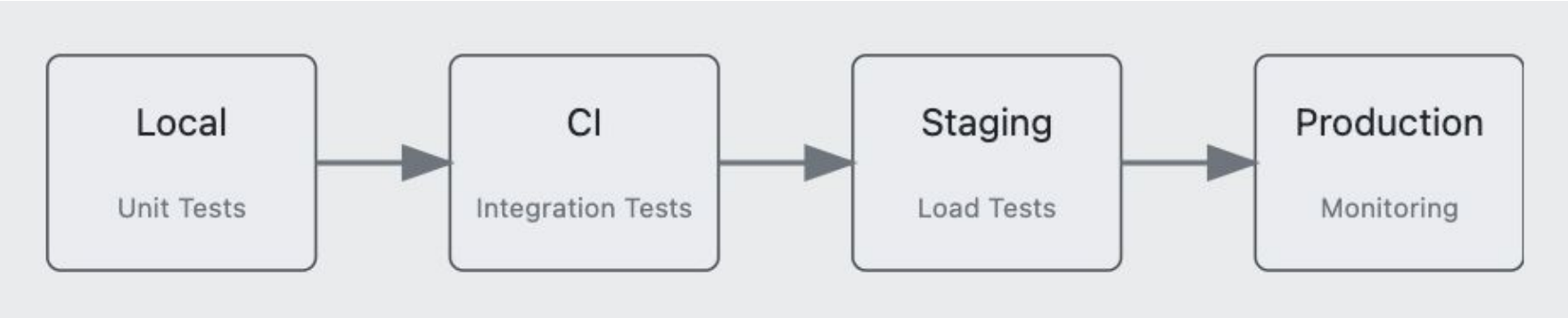
Does the forecast generalize?

Is the time split appropriate?

How does performance vary by regime?

Does it meet the intended use?

Checks across the development workflow



Examples, not restrictions: unit and integration tests run locally and in CI.

Staging adds realistic conditions; production adds live evidence.

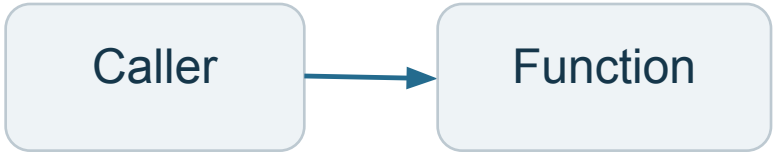
APIS AND ARCHITECTURE

APIs and systems architecture

A network boundary makes the request contract explicit.

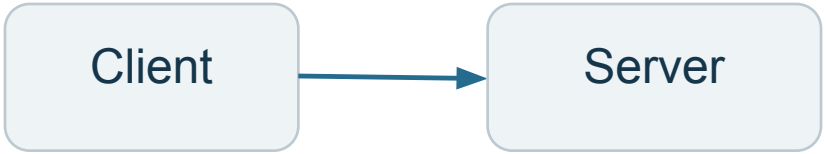
A function call and a network request

One Python process



Values passed in memory

Two processes



JSON over HTTP

Status + response body

Network failures are possible

The HTTP request and response

```
POST http://127.0.0.1:8769/predict
Content-Type: application/json

{"momentum_20d": 0.02,
 "volatility_20d": 0.15}
```

200 OK: prediction + model_version

422: request violates the input contract

500: the service could not complete inference

The prediction contract

Input features

momentum_20d

20-day return, as a fraction.

Allowed: −1 to 1.

volatility_20d

Annualized volatility, as a fraction.

Allowed: 0 to 2.

Output

prediction: return as a fraction

model_version: classroom-v1

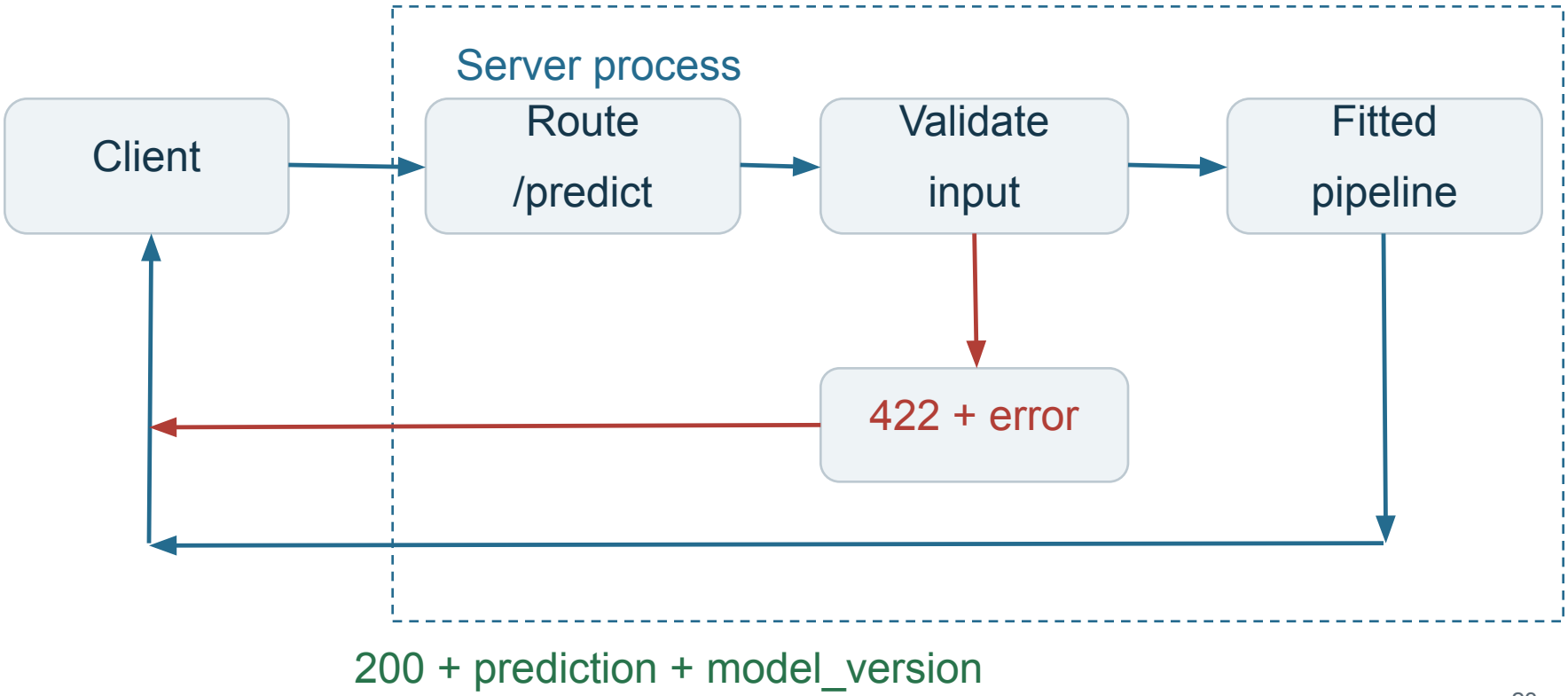
Examples

0.02 means 2%, not 0.02%.

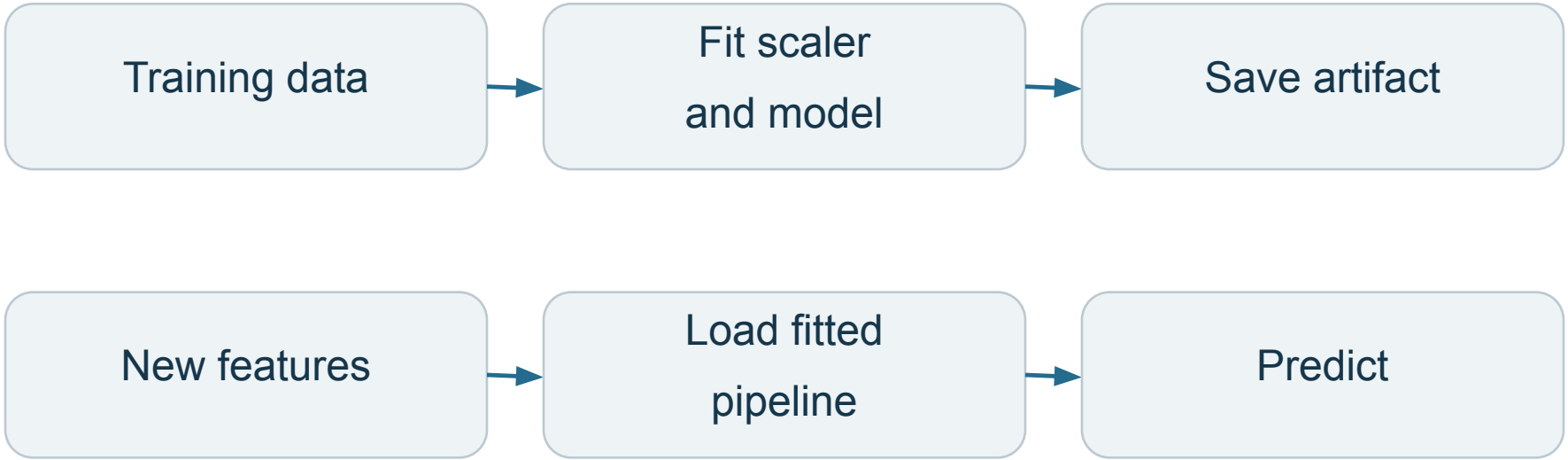
A missing field is rejected.

Unknown fields are rejected.

The request–response cycle

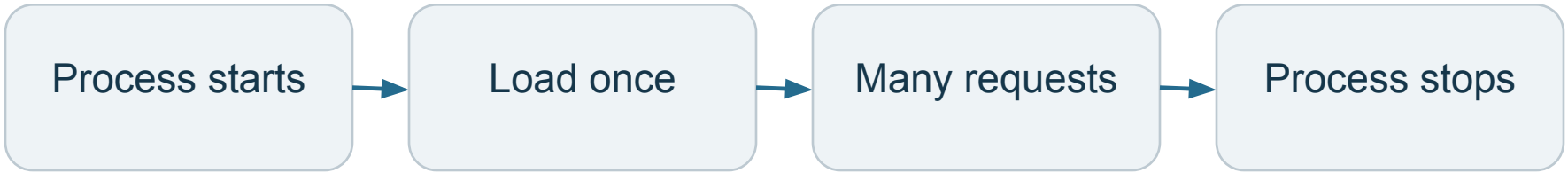


Training and serving the same pipeline



Fit during training. Reuse the fitted transforms during serving.

Model lifetime inside a service




```
@asynccontextmanager
async def lifespan(app):
    app.state.bundle = joblib.load(MODEL_PATH)
    yield
    del app.state.bundle
```

Calling and testing the service

```
direct = pipeline.predict(row)[0]

response = httpx.post(
    base_url + "/predict", json=payload
)
assert response.json()["prediction"] == approx(direct)
```

Valid input → 200 and the same prediction.

Missing feature → 422 before inference.

Batch scoring and online requests

Batch scoring

Score many rows on a schedule.

Optimize throughput and completion time.

Retry or resume a failed partition.

Example: nightly research signals.

Online requests

Answer a caller as inputs arrive.

Bound latency and concurrent load.

Return clear errors and timeouts.

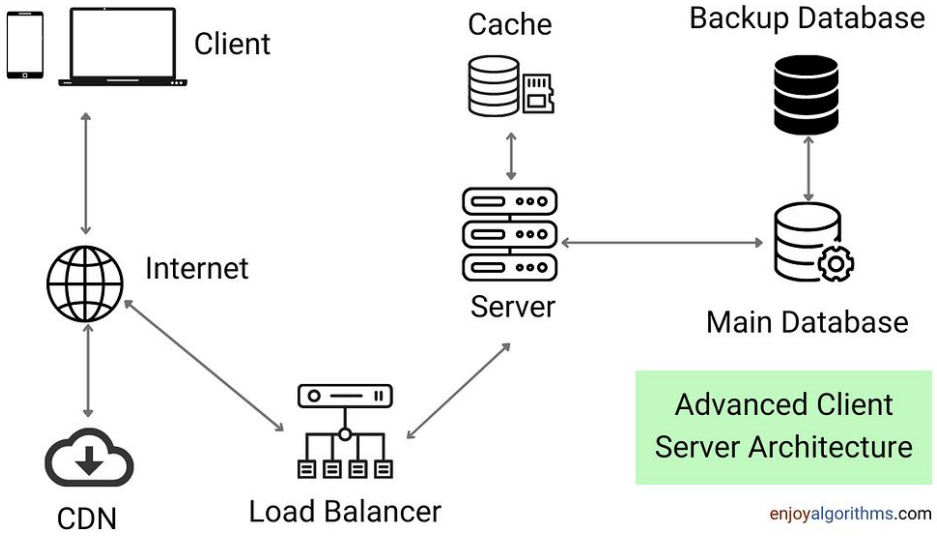
Example: an interactive prediction.

Adding workers and a load balancer

Load balancer
Routes requests to workers.

Workers
Run service processes.

Shared storage
Keeps durable state.



Add components for a measured need: load, availability, or durable state.

The memory cost of more workers

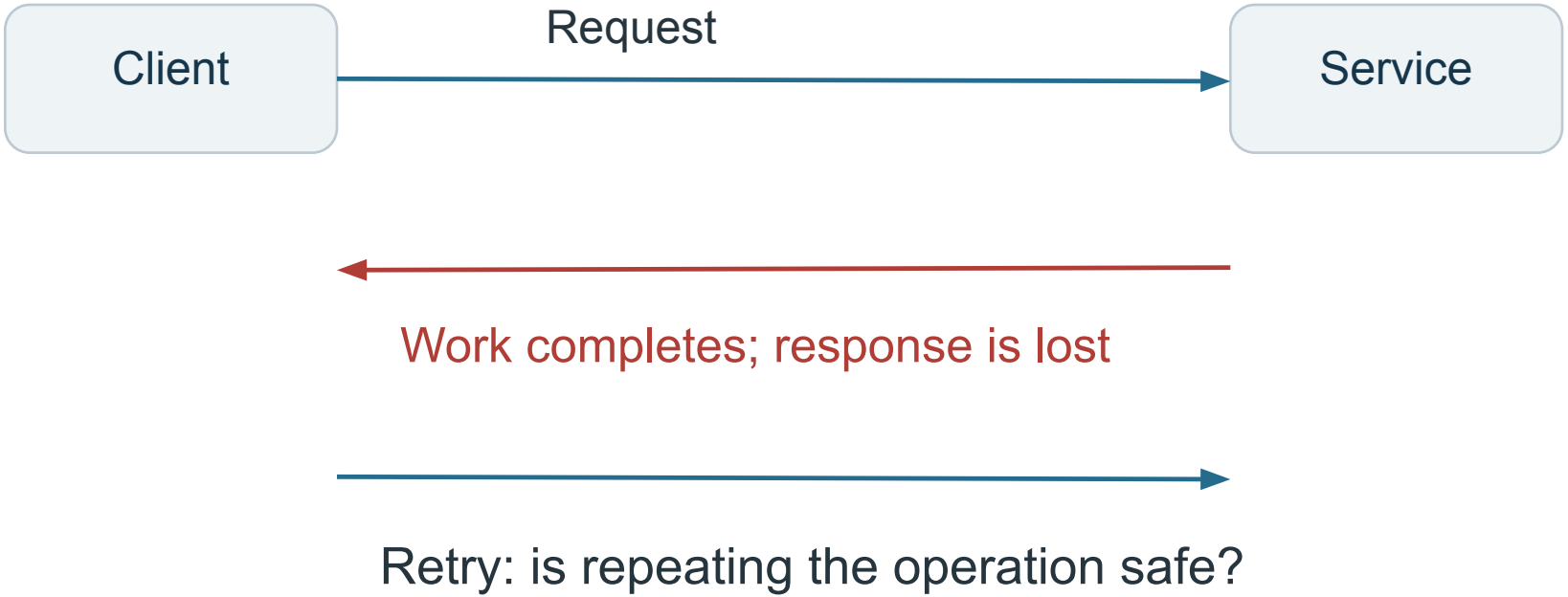
Worker 1	Worker 2	Worker 3	Worker 4
500 MB model	500 MB model	500 MB model	500 MB model

4 separate copies × 500 MB ≈ 2 GB

plus Python, libraries, request data, and buffers.

Illustrative budget: assumes one independent model copy per process.

Timeouts, retries, and repeated requests



CONTAINERS

Containers

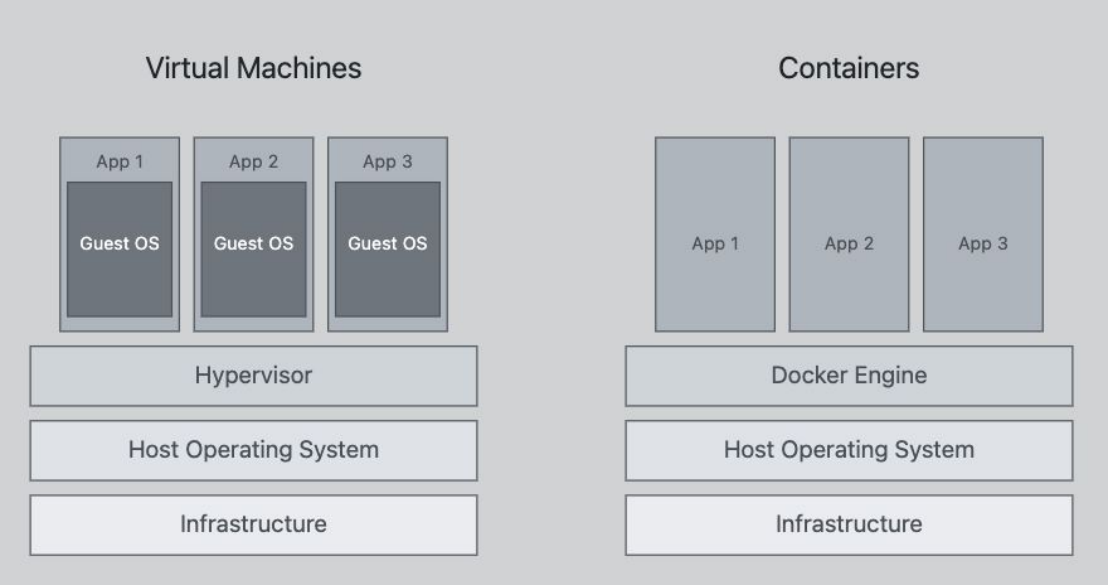
Package the runtime, code, and fitted model for repeatable execution.

Environments, containers, and virtual machines

Python environment
Isolates Python packages.

Container
Isolates processes/files; shares a kernel.

Virtual machine
Runs a guest OS.



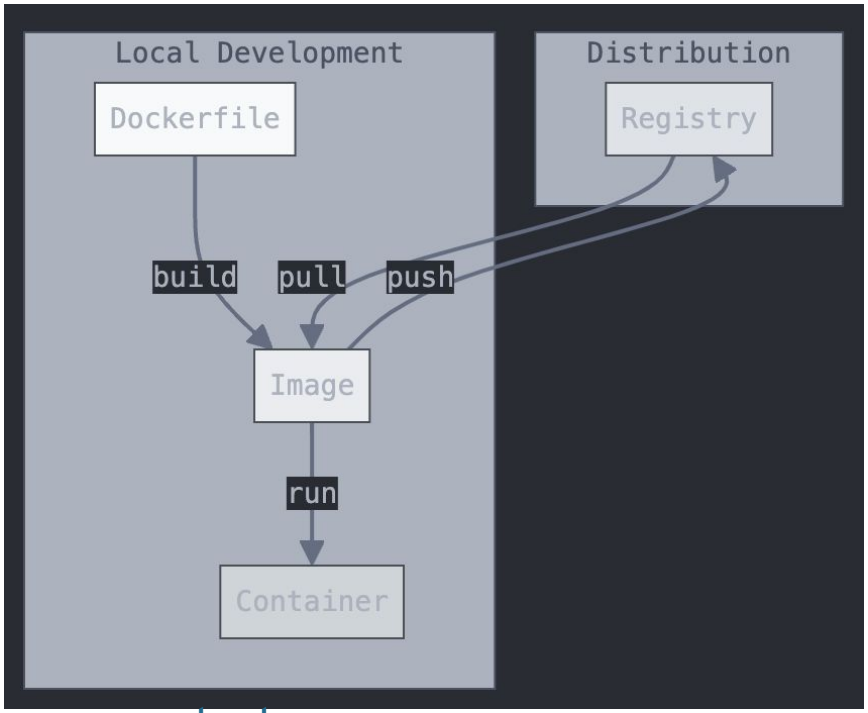
Linux containers on a Mac typically run inside a Linux VM.

Images, containers, and registries

Image
Code, model, and runtime.

Container
A running image instance.

Registry
Stores and distributes images.



Tags can move. A digest identifies exact image content.

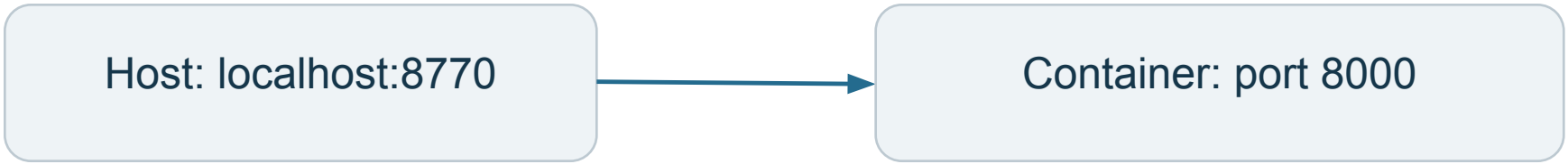
A Dockerfile for the service

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
COPY artifacts/ artifacts/
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build: install dependencies and copy files.

Run: start the API process. requirements.txt comes from uv.lock.

Ports, configuration, and stored data




```
-p 127.0.0.1:8770:8000
```

Configuration enters at runtime. Keep secrets out of the image.

Keep durable outputs in a volume or external store.

Replacing an instance should not erase business records.

The same request inside a container

```
docker build -t mfe-lesson9:classroom-v1 .

docker run --rm --name mfe-lesson9-demo \
  -p 127.0.0.1:8770:8000 mfe-lesson9:classroom-v1
```

Send the same JSON to localhost:8770/predict.

Compare the prediction and model version.

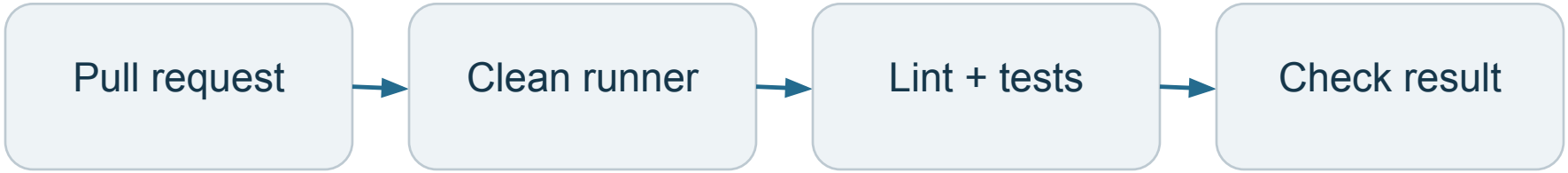
Restart the image: the packaged model is still there.

CI AND DEPLOYMENT

Continuous integration and deployment

Automated checks and versioned releases support reliable operation.

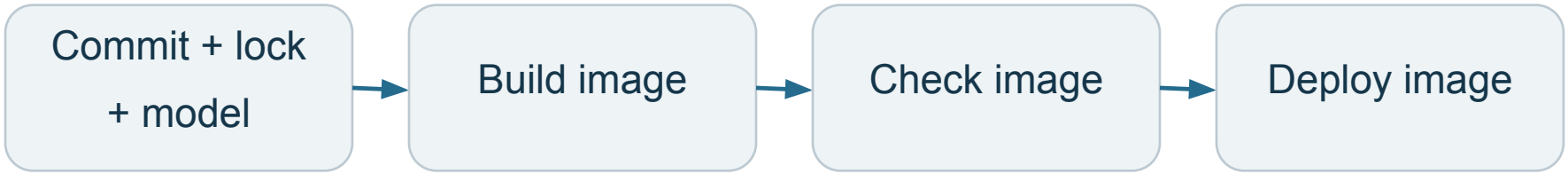
A pull request runs automated checks




```
steps:
  - run: uv sync --locked
  - run: uv run --locked ruff check .
  - run: uv run --locked ruff format --check .
  - run: uv run --locked pytest -q
```

Required status checks must be configured to block merging.

Releasing a known version

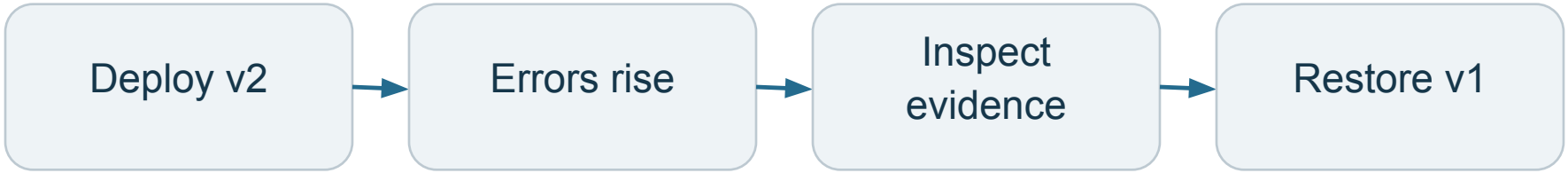


Record the image digest and model version.

Promote the same checked artifact between environments.

Retain the previous image and a recovery procedure.

Monitoring and recovery



Service: errors, latency, throughput, memory.

Data: missing fields, feature ranges, changing distributions.

Model: quality evidence when outcomes become available.

A request ID connects a failed response to its server log.

Reviewing an AI-generated change

```
# Proposed “simplification”
pipeline.fit(request_rows, request_targets)
return pipeline.predict(request_rows)
```

Contract: serving reuses a fitted pipeline.

Review: does this change refit on each request?

Evidence: test row-alone versus row-in-batch consistency.

Decision: keep training outside the prediction route.

System-design interview checkpoint

1. Valid JSON, incorrect predictions.

Which boundary and evidence do you inspect?

2. More workers, growing RAM, worse latency.

What trade-off would you measure?

3. Errors rise immediately after a release.

What would you compare and how would you recover?

Productionization practice

Serve a saved pipeline through a documented /predict contract.

Test a known result, invalid inputs, and inference failure.

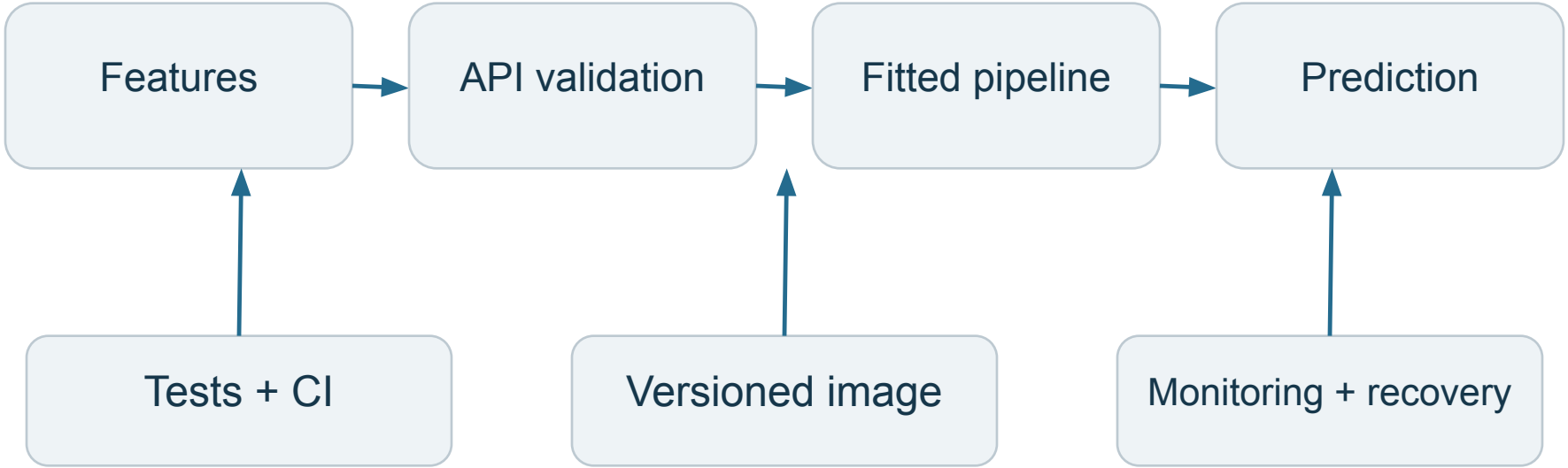
Package the service and repeat a request to the container.

Run checks in CI and show the result.

Record the model version and explain recovery.

Practice outline; submission details belong in the published assignment.

The complete system



Reference: static-analysis tools

Ruff format

Consistent layout.

Ruff check

Lint rules and import checks.

pytest

Executable behavior checks.

mypy

Optional static type analysis.

Earlier course tools

Black, isort, and flake8 served related formatting and linting roles.

A clean lint result does not prove correctness.

Reference: dependency injection

With Dependency Injection:

python

```python
from fastapi import FastAPI, Depends

app = FastAPI()

# Dependency to provide the database connection
def get_database_connection():
    return "database_connection"

@app.get("/items/")
def read_items(db=Depends(get_database_connection)):
    return {"db": db}
```

A dependency function supplies a resource.

Tests can substitute it.

Depends alone does not create a connection pool.

Reference: HTTP methods

GET

Read a representation.

POST

Submit data for processing; may create a resource.

DELETE

Remove a resource.

PUT

Replace the representation at a resource address.

PATCH

Apply a partial update.

Status and response bodies communicate the outcome.

Reference: project layout and Makefile

```
demo/
  app.py
  fees.py
  artifacts/model.joblib
  tests/
  pyproject.toml + uv.lock
  requirements.txt
  Dockerfile + Makefile
```

make test

Runs pytest.

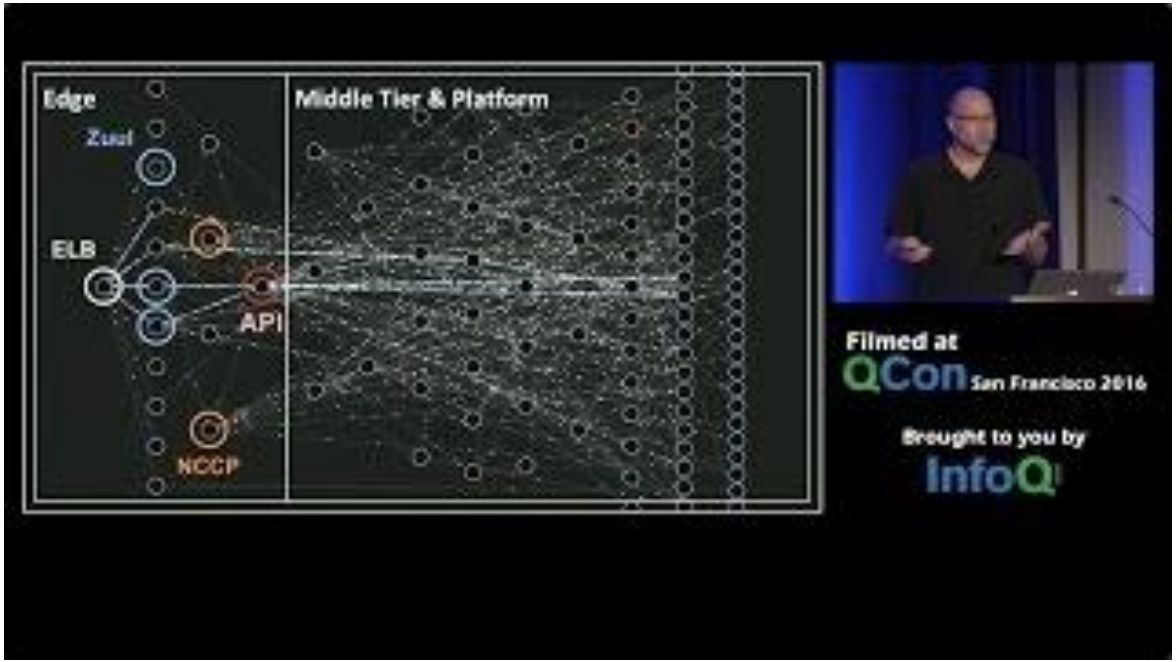
make lint

Runs lint + formatting checks.

make build / make run

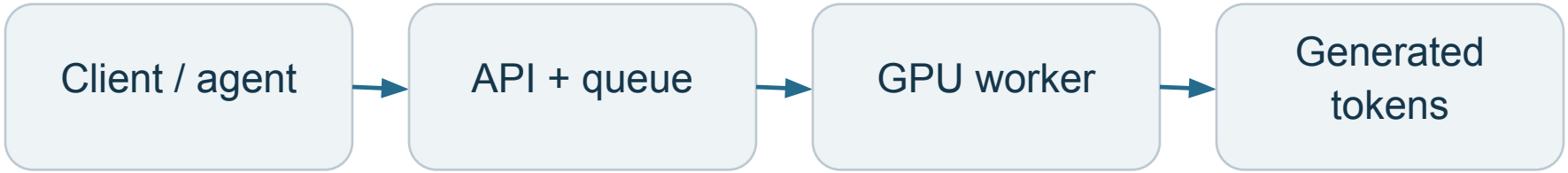
Builds / runs the container.

Reference: Netflix architecture video



Original course video · optional systems discussion

Reference: LLM serving uses the same boundaries



Model weights occupy memory before a request arrives.

Active requests also need runtime state, including KV cache.

Concurrency, batch size, context length, and latency trade off.